



Numerical Integration Calculator

v1.0

User Manual

Calculate definite integrals using four numerical methods
with step-by-step solution trails and accuracy verification

Developed by: **Gabriel Estrella** • **Daniel Christian Mendoza** • **Paul Rosal**

1. Introduction

The Numerical Integration Calculator is a desktop application built with Python and Tkinter that computes definite integrals using four classical numerical methods. It is designed for students, educators, and engineers who need quick, reliable approximations of integrals — especially for functions that cannot be integrated analytically.

Rather than returning only a final answer, the application provides a complete Solution Trail for each method: a detailed, step-by-step breakdown of every computation performed. It also cross-verifies results against SciPy's adaptive quadrature and SymPy's symbolic integration engine, giving you confidence in the accuracy of the output.

1.1 Key Features

- Four numerical integration methods computed simultaneously
- Step-by-step Solution Trail for every method
- Interactive graph visualization of the integration area
- Accuracy verification via Richardson Extrapolation, SciPy, and SymPy
- Quick Example presets to get started instantly
- Resizable, two-panel interface with scrollable controls

1.2 System Requirements

Python: 3.8 or later

Libraries: tkinter, numpy, scipy, sympy, matplotlib

Platform: Windows, macOS, or Linux

2. Interface Overview

The application window (1050 × 700 px, resizable) is divided into two main areas:

2.1 Left Panel — Setup & Controls

The left panel contains all the inputs you need to define and run a calculation.

Setup Card

- **Function $f(x)$** — Text field where you type the mathematical expression to integrate. The variable must be x .
- **Lower bound (a)** — The starting point of the integration interval.
- **Upper bound (b)** — The ending point of the integration interval.
- **Intervals (n)** — The number of subintervals used for approximation. Defaults to 100. Higher values increase accuracy.

Action Buttons

- **✦ Calculate** — Runs all four methods and displays results on the right panel.
- **Clear** — Resets all input fields and returns to the ready state.

Quick Examples Card

Four pre-built examples let you populate all fields with one click:

- x^2 on $[0, 1]$
- $\sin(x)$ on $[0, \pi]$
- e^x on $[0, 2]$
- $1/x$ on $[1, 5]$

Supported Functions Reference

Trigonometric: sin, cos, tan

Exponential/Log: exp, log (natural log), log10

Other: sqrt, abs, floor, ceil

Constants: pi ($\approx 3.14159\dots$), e ($\approx 2.71828\dots$)

Variable: Use x as the only variable

Power operator: Use $**$ or $^$ (both are accepted)

2.2 Right Panel — Results

The right panel is initially blank, showing a "Ready to Calculate" placeholder. After clicking Calculate, it displays a 2×2 grid of result cards, one for each numerical method.

Each result card shows:

- The method name and a brief description
- The computed numerical result (6 decimal places)
- A "View Solution" button to open the Step-by-Step Solution Trail
- A "Plot" button to open the graph visualization window

3. How to Use

3.1 Basic Workflow

1. **Enter a function** — Click the Function $f(x)$ field and type your expression. For example: $\sin(x)$, x^{**2} , $\exp(x)$, or $x^3 + 2x$.
2. **Set the bounds** — Enter the lower bound a and upper bound b . Make sure $a < b$.
3. **Set the number of intervals** — Enter a value for n . The default is 100. For smoother approximations, use larger values (e.g. 500 or 1000). Note that Simpson's 1/3 Rule requires n to be even, and Simpson's 3/8 Rule requires n to be a multiple of 3 — the app adjusts these automatically if needed.
4. **Click Calculate** — All four methods run simultaneously. Results appear in the right panel.
5. **Explore the results** — Click "View Solution" on any card to see the full step-by-step derivation, or "Plot" to visualize the area under the curve.

3.2 Quick Examples

If you want to explore the application without typing a function manually, use the Quick Examples in the left panel. Clicking any example row immediately fills in the function, lower bound, and upper bound fields. You can then click Calculate right away.

Function	Expression to Enter	Lower (a)	Upper (b)	Expected Answer
x^2	<code>x**2</code>	0	1	Exact answer: $1/3 \approx 0.333333$
$\sin(x)$	<code>sin(x)</code>	0	3.14159	Exact answer: ≈ 2.0
e^x	<code>exp(x)</code>	0	2	Exact answer: $e^2 - 1 \approx 6.389056$
$1/x$	<code>1/x</code>	1	5	Exact answer: $\ln(5) \approx 1.609438$

3.3 Clearing and Starting Over

Click the Clear button to reset all fields to their defaults and return the right panel to the "Ready to Calculate" state. The function field clears, bounds reset to 0 and 2, and n resets to 100.

4. Numerical Methods

The calculator uses four well-established numerical integration rules. Each one approximates the area under the curve $f(x)$ from a to b by dividing the interval into n subintervals of width $h = (b - a) / n$.

Method	Description	n Constraint
Trapezoidal Rule	Approximates area using trapezoids. Works well for smooth, nearly-linear functions. The simplest and most general method.	No special requirement
Simpson's 1/3 Rule	Uses parabolic segments for higher accuracy. Generally the most accurate method for smooth functions.	n must be even
Midpoint Rule	Evaluates the function at the midpoint of each subinterval. Surprisingly accurate for oscillating functions.	No special requirement
Simpson's 3/8 Rule	Uses cubic polynomials for approximation, offering high precision on smooth curves.	n must be a multiple of 3

4.1 Trapezoidal Rule

The Trapezoidal Rule approximates the area under the curve by connecting consecutive function values with straight lines (trapezoids). It applies a coefficient of 1 to the endpoints and 2 to all interior points.

Formula:

$$\text{Integral} \approx (h/2) \times [f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) + f(x_n)]$$

Note: This is the most general-purpose method and a good starting point for any function.

4.2 Simpson's 1/3 Rule

Simpson's 1/3 Rule fits parabolas through consecutive groups of three points, yielding significantly better accuracy than the Trapezoidal Rule for smooth functions. The coefficient pattern alternates as 1, 4, 2, 4, 2, ..., 4, 1.

Formula:

$$\text{Integral} \approx (h/3) \times [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 4f(x_{n-1}) + f(x_n)]$$

Note: n must be even. If you enter an odd n , the app automatically increments it to the next even number.

4.3 Midpoint Rule

Instead of evaluating $f(x)$ at the boundaries of each subinterval, the Midpoint Rule evaluates at the center (midpoint) of each subinterval. This avoids endpoint values and can be more accurate for oscillating functions.

Formula:

$$\text{Integral} \approx h \times \sum f(a + (i + 0.5) \times h) \quad \text{for } i = 0, 1, \dots, n-1$$

4.4 Simpson's 3/8 Rule

Simpson's 3/8 Rule is a higher-order variant that fits cubic polynomials through groups of four consecutive points. The coefficient pattern is 1, 3, 3, 2, 3, 3, 2, ..., 3, 3, 1.

Formula:

$$\text{Integral} \approx (3h/8) \times [f(x_0) + 3f(x_1) + 3f(x_2) + 2f(x_3) + \dots + f(x_n)]$$

Note: n must be a multiple of 3. If your value does not satisfy this, the app rounds up to the next valid multiple.

5. Solution Trail

The Solution Trail is the most educational feature of the application. Clicking "View Solution" on any result card opens a dedicated window that walks through every arithmetic step involved in the calculation.

5.1 What the Solution Trail Contains

Each Solution Trail follows the same structure:

6. **Given** — Displays the function, lower bound a , upper bound b , and number of intervals n .
7. **Subinterval Width** — Shows the formula $h = (b - a) / n$ and the computed value of h .
8. **Formula Reference** — Displays the exact formula for the chosen method, including coefficient patterns.
9. **Function Evaluations** — Lists each x -point, its $f(x)$ value, and its coefficient. Up to 8 points are shown explicitly; remaining points are summarized.
10. **Weighted Sum** — Displays the intermediate weighted total before the final multiplication.
11. **Final Result** — Multiplies the weighted sum by the appropriate factor ($h/2$, $h/3$, $3h/8$, or h) to produce the answer.

5.2 Verification & Accuracy Audit

After the calculation steps, the Solution Trail automatically runs three independent checks:

- **Richardson Extrapolation** — Repeats the calculation with $2n$ intervals to estimate the approximation error. A large difference between the n and $2n$ results indicates that n may be too small.
- **SciPy Adaptive Quadrature** — Uses `scipy.integrate.quad` to compute a highly accurate numerical reference value. The absolute and relative errors are reported.
- **SymPy Symbolic Integration** — Attempts to compute the exact symbolic antiderivative. If successful, shows the antiderivative, the exact numerical value, and the error against it.

At the end, the trail displays either:

- ✓ **RESULT: HIGH ACCURACY** — Absolute error < 0.0001 (compared to SciPy reference).
- ⚠ **RESULT: CONSIDER INCREASING n** — Error exceeds 0.0001 ; try a larger n .

5.3 Using the Solution Trail Window

- Scroll vertically and horizontally to read long outputs
- The window is resizable — drag its edges to make it wider
- Click the Close button or the window's X button when done

6. Graph Visualization

Clicking the "Plot" button on any result card opens the Graph Visualization window. This shows a Matplotlib chart that makes the integration geometry concrete and visual.

6.1 What the Plot Shows

- The function curve $f(x)$ drawn as a smooth line
- The shaded area representing the approximated integral
- The individual geometric shapes used by the method (trapezoids, rectangles, or parabolic segments), drawn as translucent patches in the method's accent color
- Labeled axes, a title, and a grid for readability

6.2 Interacting with the Plot

The Matplotlib toolbar appears at the bottom of the plot window, giving you standard controls:

- Pan (drag) and Zoom (scroll or select region) to explore different parts of the curve
- Home button to reset the view
- Save button to export the chart as a PNG image file

7. Understanding Results & Choosing Parameters

7.1 Choosing n (Number of Intervals)

The default value of $n = 100$ is sufficient for most smooth functions. Here is a general guide:

n Value	When to Use	Trade-off
10 – 50	Quick estimates, highly irregular functions	Lower accuracy
100 (default)	Most standard smooth functions	Good balance
500 – 1000	Functions that oscillate rapidly	Slower compute
1000+	Verification runs or near-singular functions	May be slow

7.2 Interpreting the Accuracy Audit

If the Solution Trail shows $\triangle$ CONSIDER INCREASING n , try doubling n and recalculating. If the result changes significantly between n and $2n$, your original estimate was insufficiently accurate. Keep increasing n until the result stabilizes.

For functions with singularities (e.g., $1/x$ near 0, $\tan(x)$ near $\pi/2$), ensure your interval $[a, b]$ does not include the problematic point. Set a bound that avoids it.

7.3 Which Method Should I Use?

In most cases, all four methods will converge to the same result as n increases. However:

- For smooth, well-behaved functions → Simpson's 1/3 Rule typically gives the best accuracy per interval.
- For functions that are difficult to differentiate or highly irregular → Trapezoidal Rule is the safest choice.
- For functions that oscillate around zero → Midpoint Rule often performs better than Trapezoidal.
- When you need a third-order method → Simpson's 3/8 Rule is a good alternative to Simpson's 1/3.

8. Error Messages & Troubleshooting

Error / Warning	What to Do
Please enter a function	The function field is empty. Type a valid expression (e.g. $\sin(x)$).
Invalid a, b, or n	One of the bound or interval fields contains non-numeric text. Check that a and b are numbers and n is a whole number.
a must be < b	The lower bound must be strictly less than the upper bound. Swap the values if needed.
n must be ≥ 2	Enter a value of at least 2 for n. Smaller values produce meaningless approximations.
Function error: ...	The expression could not be evaluated. Check for unsupported functions, missing parentheses, or division by zero at the test point. See Section 9 for supported function names.
 CONSIDER INCREASING n	The accuracy check found a non-trivial error. Double n and try again until the result stabilizes.

9. Function Syntax Reference

Functions are entered using standard Python/math notation with x as the only variable. The following are available:

9.1 Arithmetic Operators

```
x + 2      # Addition
x - 5      # Subtraction
x * 3      # Multiplication
x / 4      # Division
x**2 or x^2 # Exponentiation (both are accepted)
```

9.2 Mathematical Functions

```
sin(x)      # Sine (radians)
cos(x)      # Cosine (radians)
tan(x)      # Tangent (radians)
exp(x)      # e raised to the power x
log(x)      # Natural logarithm (base e)
log10(x)    # Logarithm base 10
sqrt(x)     # Square root
abs(x)      # Absolute value
floor(x)    # Round down to nearest integer
ceil(x)     # Round up to nearest integer
```

9.3 Constants

```
pi          #  $\pi \approx 3.14159265358979$ 
e           # Euler's number  $\approx 2.71828182845905$ 
```

9.4 Complex Expressions

You can combine functions and operators in any valid mathematical way:

```
x**2 + sin(x)
exp(-x**2)
sqrt(1 - x**2)
log(x) / x
3*x**3 - 2*x + 1
```

Note: Always use x as the variable name. Other variable names (t , u , y) are not supported and will produce a Function error.

10. About the Application

Numerical Integration Calculator v1.0 was developed as an educational tool to help students understand and apply the fundamental methods of numerical integration. It combines a user-friendly desktop interface with rigorous back-end verification to make learning numerical analysis more accessible.

Developed by:

- Gabriel Estrella
- Daniel Christian Mendoza
- Paul Rosal

Built with Python, Tkinter, NumPy, SciPy, SymPy, and Matplotlib.

Version: 1.0

Interface: Tkinter (Python standard library)

Computation: NumPy, SciPy

Symbolic math: SymPy

Visualization: Matplotlib (TkAgg backend)